

Regression for Yes/No Outcomes: Logit

MKT 566 · Fall 2026

Davide Proserpio

USC Marshall School of Business

Before We Start

Housekeeping

- **Homework 2** (box office, OLS and logit) is out, due **Thursday, Oct. 8**. Everything it needs is in Thursday's deck and today's
- **Group project**: proposal presentations on **Oct. 13 and 15**
- Today: **logit** for about 40 minutes, then the **Airbnb case** on your laptop (aim to finish Task 5 in class; report due **Sunday, Sept. 27**)
- Thursday: **clustering**

From “how much” to “whether”

Thursday’s regressions predicted a **number**: sales, price. Many marketing questions are **yes or no**:

- Will this customer **buy**? **Churn**? **Click** the ad? **Open** the email?
- Will this loan **default**? Will this listing be a **hit**?

Today you learn to

1. See why a straight line is the wrong tool for a 0/1 outcome
2. Read a **logit** model: sign, odds, and predicted probabilities
3. Run it in R with `glm()` and put it in a **stargazer** table

What stays the same

- Predictors, dummies, base levels, stars, “holding the others fixed”: all as in OLS
- The AI writes the code. You choose the model and read the result

R script for today

In the [code/](#) folder on the course website, next to Thursday's:

1. [w4-2-logit-class.R](#): every chart and model in today's deck
2. [data/airbnb.csv](#): the same Airbnb listings

[w4-code.zip](#) now contains both scripts and the data. If you downloaded it on Thursday, download it again.

A Yes/No Outcome

Our outcome: is a listing a “gem”?

An investor wants listings that are **both** well rated **and** actually booked. Define a **gem** as a listing with a rating of **4.5 or more** and **more than 20 reviews**:

```
1 airbnb[, gem := as.integer(star_rating >= 4.5 & reviews_count > 20)]
2 table(airbnb$gem)
```

```
      0      1
40460 10358
```

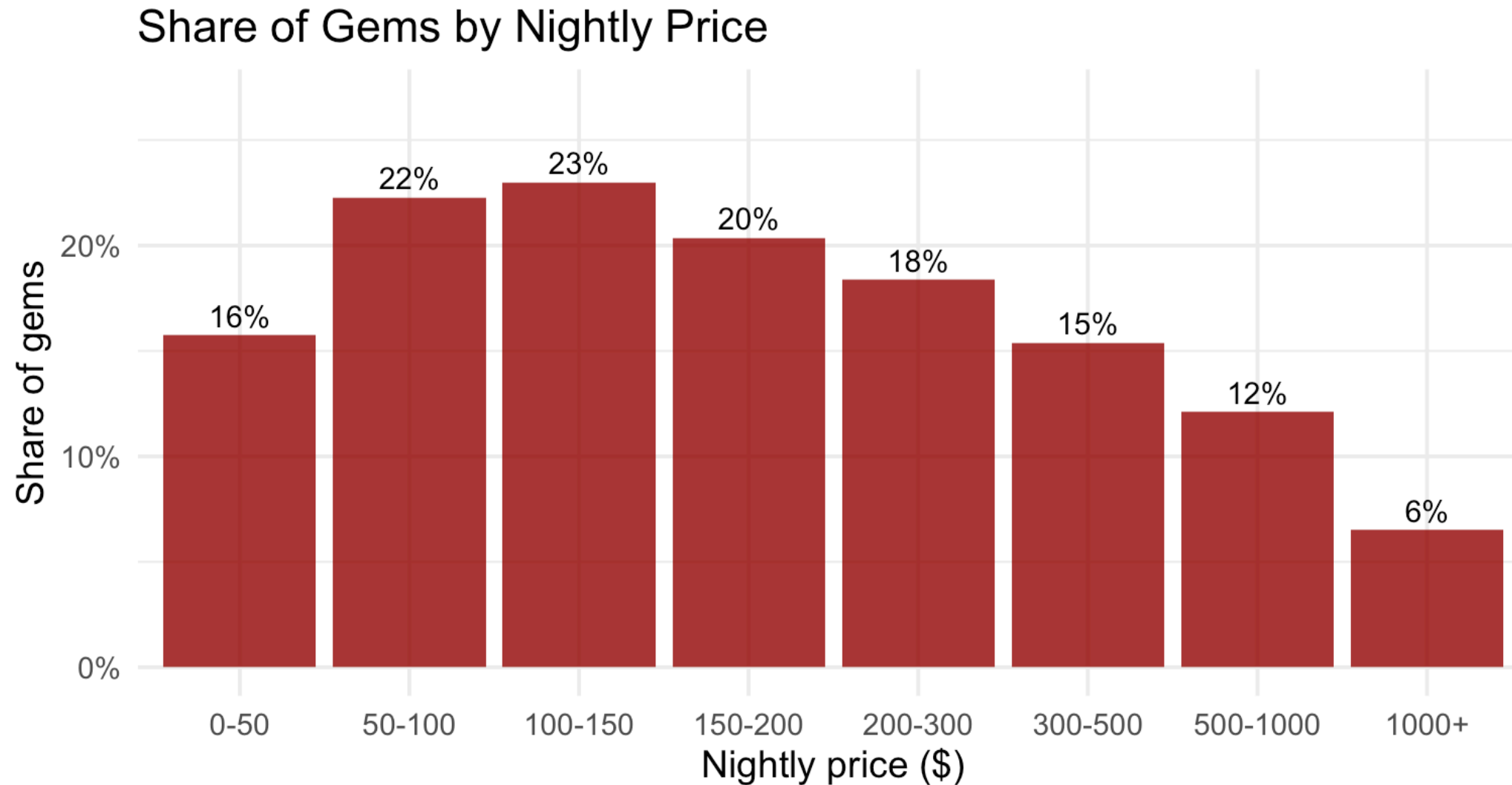
```
1 mean(airbnb$gem)
```

```
[1] 0.2038254
```

- **gem** is a **dummy**: 1 for a gem, 0 otherwise. About **20%** of listings are gems
- The question: **what makes a listing more likely to be a gem?** Start with price

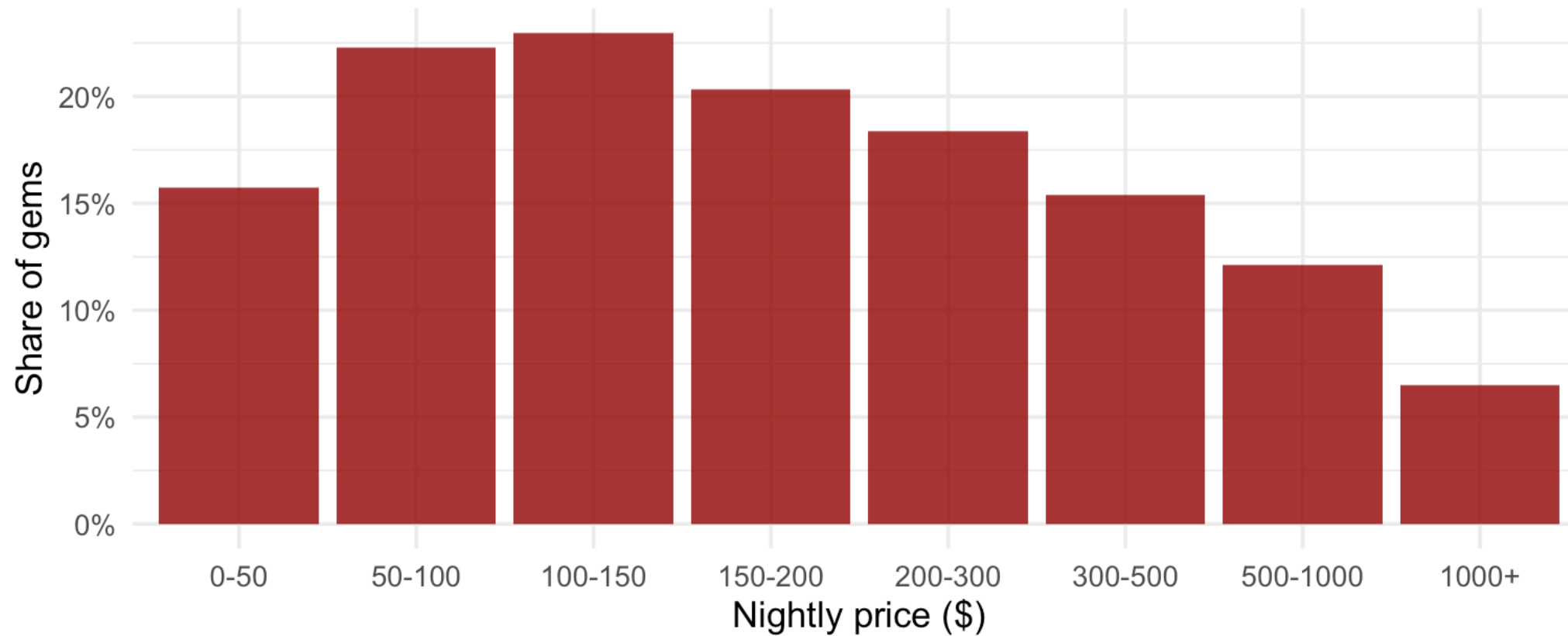
The **&** means both conditions must hold; **as.integer()** turns TRUE/FALSE into 1/0, as in Task 7 of the case.

Share of gems by price



Same idea as the “average by group” charts of week 3, with a 0/1 variable: the average of a dummy is a **share**. What do you see?

Share of gems by price: reading it

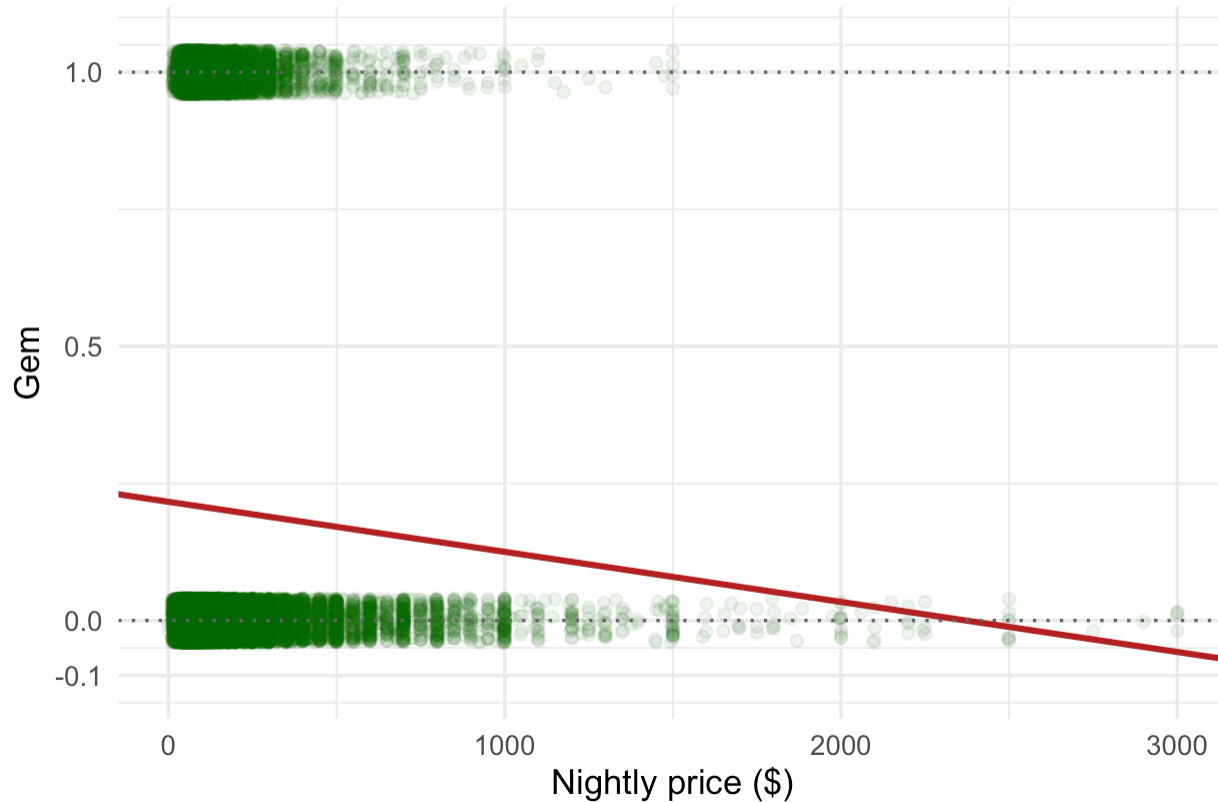


- Gems are **rarer as price rises**: 22–23% of listings under \$150, 12% between \$500 and \$1,000, 6% above \$1,000. The very cheapest bin is also lower
- So the **probability** of being a gem depends on price. Can we model that probability with a regression, the way we modeled price on Thursday?

Try the tool we have: a straight line

```
1 lpm <- lm(gem ~ price, data = airbnb) # OLS, with a 0/1 outcome
```

Gem (0/1) vs. Price, with the OLS line



- The “linear probability model”: the line is the **predicted probability** of being a gem
- Intercept **0.217**: a free listing has a 21.7% chance. Slope **-0.00009**: each extra dollar lowers it by 0.009 percentage points; \$100 more, by about **0.9 points**
- Reasonable so far. Now look at the right end of the line

Where the line breaks

```
1 pred <- predict(lpm)
2 sum(pred < 0)           # listings with a predicted probability below zero
```

```
[1] 22
```

```
1 round(min(pred), 2)     # the lowest one
```

```
[1] -0.7
```

- The line crosses zero at a price of about **\$2,372** and keeps going: a \$10,000 listing gets a “probability” of **-70%**
- A line cannot bend. But a probability must stay **between 0 and 1**, so the true relationship must **flatten** as it approaches 0 (and 1)
- A second problem: the line says \$100 more changes the probability by the same 0.9 points whether you start at 20% or at 2%

For a yes/no outcome we need a curve that is **linear in the middle** and **flattens at both ends**. That curve is the logistic function, and the model built on it is the **logit** (logistic regression).

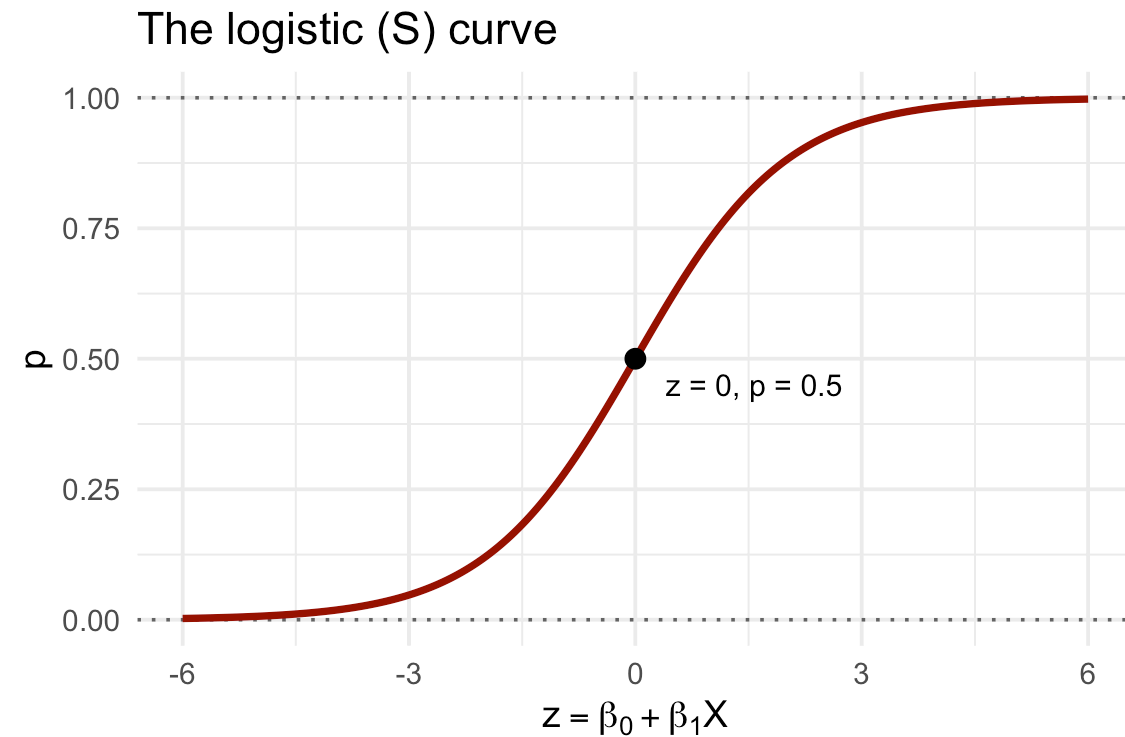
Logistic Regression

The fix: squash the line into 0–1

Keep the linear part, $z = \beta_0 + \beta_1 X$, and pass it through the **logistic function**:

$$p = P(Y = 1 \mid X) = \frac{1}{1 + e^{-z}}$$

- Any z , from $-\infty$ to $+\infty$, comes out as a number between **0 and 1**
- $z = 0$ gives $p = 0.5$. Large positive z : close to 1. Large negative: close to 0



Odds and log-odds

Rearranging the same equation gives the form the coefficients live in:

$$\frac{p}{1-p} = e^z \qquad \log\left(\frac{p}{1-p}\right) = z = \beta_0 + \beta_1 X$$

- $\frac{p}{1-p}$ is the **odds**: the chance it happens against the chance it does not. $p = 0.2$ means odds of 0.25, “one to four”. $p = 0.5$ means odds of 1, “even”
- The **log of the odds** is a **straight line** in X . That is what “logit” means, and why everything from OLS carries over: predictors, dummies, base levels, standard errors, stars

A logit coefficient β_1 is the change in the **log-odds** for one more unit of X . Nobody thinks in log-odds, so the next slide converts.

Reading a logit coefficient

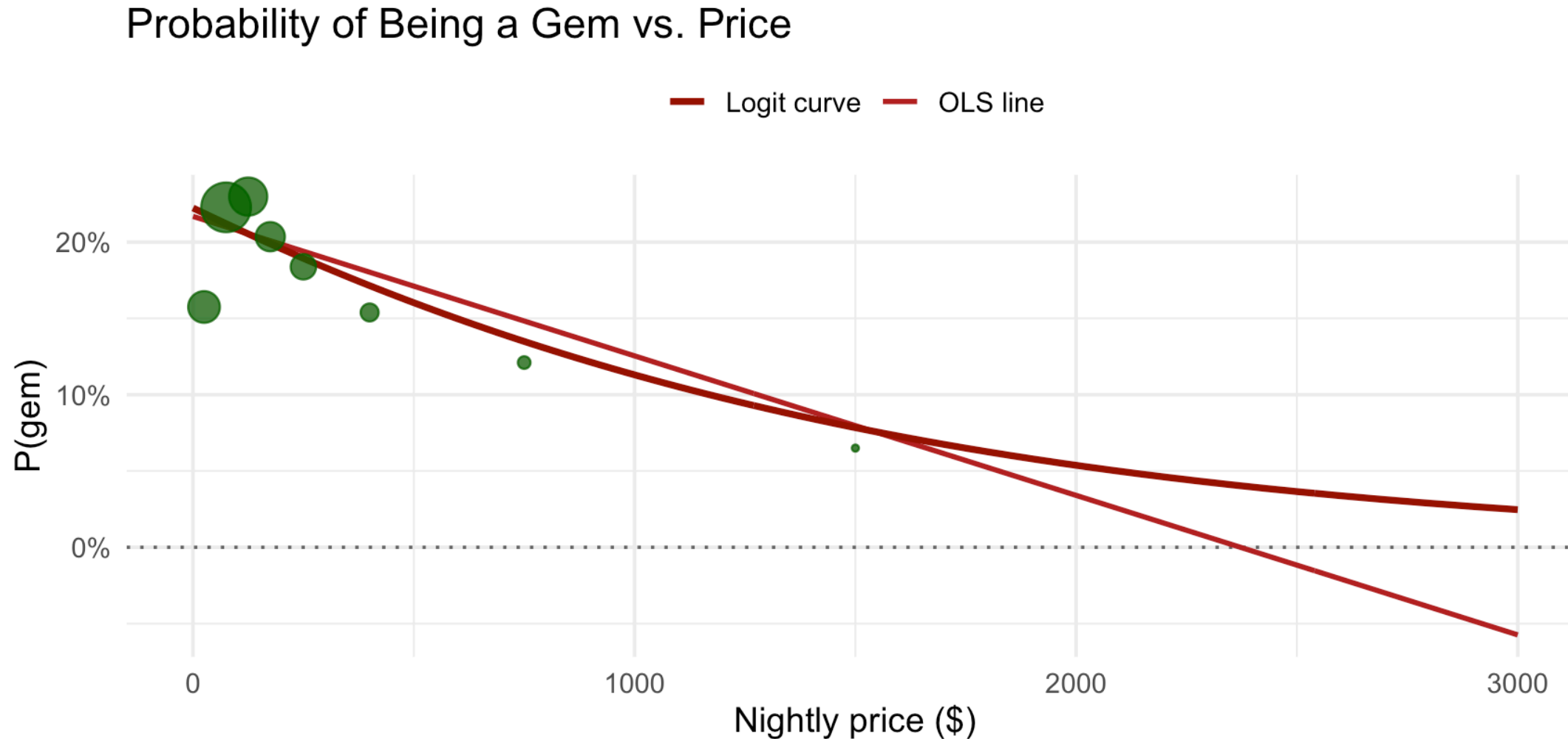
Because $\text{odds} = e^z$, one more unit of X **multiplies** the odds by e^{β_1} . Three ways to say the same thing:

Reading	How	Example with $\beta_1 = -0.0008$ per dollar
Sign and stars	as in OLS	negative, significant: pricier listings are less likely to be gems
Odds ratio	e^{β_1}	$e^{-0.0008} = 0.9992$: each dollar multiplies the odds by 0.9992
Percent change in odds	$(e^{\beta_1} - 1) \times 100$	-0.08% per dollar; for \$100, $e^{-0.08} - 1 = -7.8\%$

- For small β_1 (below about 0.2), $\beta_1 \times 100$ is a fine approximation of the percent change in odds
- To change the unit, scale the coefficient **before** exponentiating: $100 \times \beta_1$ for a \$100 change

The percent change is in the **odds**, not in the probability. A 50% increase in odds takes a probability of 20% to 27%, not to 30%.

Logit on price: the curve, on the data



Green dots: the observed share of gems in each price bin (sized by number of listings). Which model fits the dots, and what happens past \$2,000?

The curve: reading it

```
1 lg1 <- glm(gem ~ price, data = airbnb, family = binomial)
2 round(coef(lg1), 5)
```

```
(Intercept)      price
   -1.25198    -0.00081
```

- The logit curve never goes below zero: at \$2,372, where the line hits zero, the curve still says about **8%**; at \$10,000, essentially zero
- In the range where most listings are (\$50–\$300) the two agree almost perfectly. The difference is in the tails, and in the guarantee
- Coefficient on price: -0.00081 . Each dollar multiplies the odds of being a gem by $e^{-0.00081} = 0.9992$. For \$100: odds $\times 0.92$, an **8% drop in the odds**

Predicted probabilities are the reading a manager understands. `predict(..., type = "response")` gives them: \$50 $\rightarrow$ 21.5%, \$200 $\rightarrow$ 19.6%, \$500 $\rightarrow$ 16%, \$1,000 $\rightarrow$ 11%, \$5,000 $\rightarrow$ 0.5%.

How it is estimated, and how to judge the fit

Estimation. OLS picks the line with the smallest squared errors. Logit picks the coefficients that make the observed 0s and 1s **most likely** (**maximum likelihood**). R does it in a fraction of a second; you never need the formula.

Log-likelihood. How likely the data is under the model. Always negative; **closer to zero is better**. Use it to compare models on the same data, the way you used R^2 on Thursday.

For the price-only model: log-likelihood $-25,650$ vs. $-25,697$ without price; pseudo- R^2 0.002 . Price matters, and explains very little, the same lesson as Thursday.

There is no R^2 . Alternatives:

- **Pseudo- R^2** (McFadden): how much the log-likelihood improves over a model with no predictors. 0 to 1, but values of 0.1–0.3 are already decent
- **Accuracy:** predict “gem” when $p > 0.5$ and count the hits. Beware the base rate: saying “not a gem” for **everyone** is right **80%** of the time here. Week 9 (classifiers) covers this properly

Logit in R

One function: `glm()`

`glm()` works like `lm()`, with `family = binomial` to ask for a logit:

```
1 m_logit <- glm(gem ~ price + guests_included + city + room_type, data = airbnb, family = binomial)
2 stargazer(m_logit, type = "text", omit.stat = c("f", "ser", "aic", "bic"),
3           digits = 4, no.space = TRUE, single.row = TRUE)
```

```
=====
                        Dependent variable:
                        -----
                                gem
-----
price                -0.0017*** (0.0001)
guests_included      0.1119*** (0.0079)
cityBoston           0.1674*** (0.0405)
cityLos Angeles      0.1617*** (0.0293)
cityMiami            -0.1955*** (0.0416)
room_typePrivate room -0.0072 (0.0265)
room_typeShared room -0.7068*** (0.0642)
Constant             -1.3935*** (0.0349)
-----
Observations                50,818
Log Likelihood             -25,420.2300
=====
Note:                *p<0.1; **p<0.05; ***p<0.01
```

Standard errors in parentheses next to each coefficient. Read the signs, then the sizes. Which predictor has the largest effect?

Reading the table

Coefficient	Odds multiply by	Reading (holding the others fixed)
price −0.0017	$e^{-0.17} = 0.84$ per \$100	\$100 more: odds of being a gem 16% lower
guests_included 0.11	1.12 per guest	one more guest included: odds 12% higher
Boston 0.17, Los Angeles 0.16	1.18, 1.18	vs. Austin: odds about 18% higher
Miami −0.20	0.82	vs. Austin: odds 18% lower
Private room −0.01, not significant	0.99	same odds as an entire home
Shared room −0.71	0.49	odds halved vs. an entire home

- The **largest** effect is the shared-room dummy, not price: a \$100 price difference moves the odds less than the room type does
- Log-likelihood −25,420, vs. −25,697 with no predictors

From odds to probabilities

Odds ratios compare. Probabilities decide. `predict()` with `type = "response"` gives the probability for any listing you describe:

```
1 examples <- data.frame(price = c(100, 100, 300, 100), guests_included = 2,  
2                           city = c("Austin", "Boston", "Austin", "Austin"),  
3                           room_type = c("Entire home", "Entire home", "Entire home", "Shared room"))  
4 examples$p_gem <- round(predict(m_logit, examples, type = "response"), 3)  
5 examples
```

	price	guests_included	city	room_type	p_gem
1	100	2	Austin	Entire home	0.208
2	100	2	Boston	Entire home	0.236
3	300	2	Austin	Entire home	0.157
4	100	2	Austin	Shared room	0.114

- An entire home in Austin at \$100 for 2 guests: **21%** chance of being a gem. The same listing in Boston: **24%**. At \$300 instead of \$100: **16%**. As a shared room: **11%**
- This is the sentence for the investor: “moving the same property from Austin to Boston raises the chance it becomes a gem from 21% to 24%”

Odds ratios for a table, probabilities for a decision. Always compute a few predicted probabilities for realistic cases before you write a recommendation.

OLS and logit, side by side

	(Thursday)	(today)
Outcome	a number (price, sales)	yes/no (gem, buy, churn, click)
Model	$Y = \beta_0 + \beta_1 X$	$\log \frac{p}{1-p} = \beta_0 + \beta_1 X$
R	<code>lm(y ~ x)</code>	<code>glm(y ~ x, family = binomial)</code>
β_1 means	one more unit of $X \rightarrow \beta_1$ more units of Y	one more unit of $X \rightarrow \text{odds} \times e^{\beta_1}$
Predictions	<code>predict(m)</code>	<code>predict(m, type = "response")</code> gives probabilities
Fit	R^2 , residual standard error	log-likelihood, pseudo- R^2
Estimation	least squares	maximum likelihood
Same in both	predictors, logs of X , dummies and base levels, SE, stars, “holding the others fixed”, not causation	

Still not causation

Cheaper listings are more likely to be gems. Three stories, as always:

1. Lower prices bring more bookings, hence more reviews, hence gem status
2. Hosts of listings that are already popular have no need to discount, and hosts of struggling ones cut prices: the causality runs the other way
3. Gems are **older** listings (20+ reviews take time), and older listings were priced in an earlier, cheaper market

“A \$100 higher price is **associated with** 16% lower odds of being a gem.” The model holds fixed guests, city, and room type. It does not hold fixed listing age, photos, or host effort.

The Airbnb Case

The case

The handout ([w4-airbnb-case.html](#)) works like RateBeer, with less help: an exact prompt for Task 1, skeletons for Tasks 2–3, and only the goal from Task 4 on.

The client is an **investor** deciding where to buy and what to list:

1. **Tasks 1–3**: look at the data, price vs. reviews with and without logs, the three models side by side (Thursday, applied)
2. **Tasks 4–5**: more predictors, then city as a factor and as hand-made dummies
3. **Tasks 6–7**: a revenue proxy, room type, a quality dummy
4. **Task 8**: three recommendations, one of them a limitation

Under every task: **predict** before you run, **explain back** one line after.

Aim to finish **Task 5** in class. Deliverable, same as always: an **R Markdown report knitted to HTML** plus [ai-log.md](#), due **Sunday, Sept. 27, 11:59 pm**. Data: [airbnb-case-data.zip](#).

Wrap-up

Code and data to reproduce today's models, on the course website:

- [w4-code.zip](#): both week 4 scripts and the data
- [w4-2-logit-class.R](#): every chart and model shown today
- [The Airbnb case handout](#) and its [data](#)

Required reading: Chapter [3.6](#) of *R for Marketing Students*. Optional: Chapter 9.2 of *R for Marketing Research and Analytics*.

Questions?